

Karigo

A voice-first, bilingual (English / Roman-Urdu) AI booking platform for on-demand services in Pakistan.

Karigo lets a user describe a service need in natural speech — "Mujhe kal subah G-13 mein AC technician chahiye" — and orchestrates a multi-agent pipeline that parses the request, resolves the location, finds and ranks providers, dispatches the booking to the provider over WhatsApp with tap-to-respond buttons, and tracks the conversation through to a confirmed visit. Every model call, API source, and state transition is logged and surfaced live to the user as the workflow runs.

Table of contents

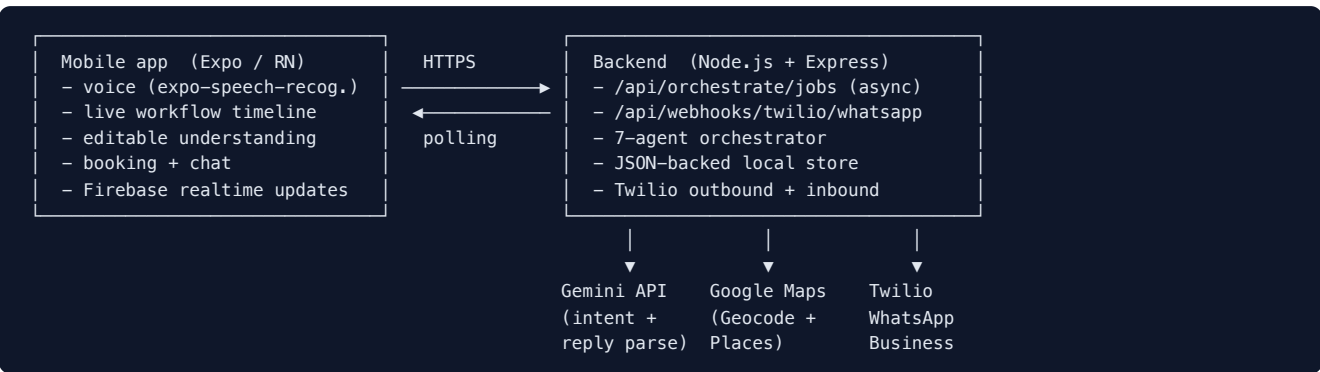
- 1. [Solution overview](#solution-overview)
- 2. [Architecture](#architecture)
- 3. [Agent pipeline](#agent-pipeline)
- 4. [Mock vs. real APIs](#mock-vs-real-apis)
- 5. [Integrations](#integrations)
- 6. [Data model & storage](#data-model--storage)
- 7. [Repository layout](#repository-layout)
- 8. [Running locally](#running-locally)
- 9. [Environment variables](#environment-variables)

Solution overview

Karigo's core design bets:

- **Conversational, not transactional.** No dropdowns or category trees. The user speaks or types in any mix of English and Urdu; the agent pipeline extracts structured intent.
- **Transparent AI, not a black box.** Every agent step is streamed to the client as a typed event (`agent` , `tool` , `source` , `status` , `summary` , `output`) and rendered as a live timeline with the *Via* source (`Google Gemini` , `Google Places` , `Google Maps` , etc.) attributed on each card.
- **Editable by default.** After the workflow runs, the user can correct any extracted field (service, sector, time) and re-run; the pipeline is idempotent.
- **Meet providers where they already are.** Providers don't install an app. Karigo dispatches bookings over WhatsApp via Twilio Content Templates with native Accept / Reject quick-reply buttons, and parses free-form replies (English / Urdu / Roman-Urdu) with a hybrid rules + Gemini classifier.
- **One real backend, swappable adapters.** Every external dependency (LLM, geocoder, places, distance) goes through a `runWithAdapter` shim so the same code path runs against the real API or a deterministic local stub — important for offline demos and tests.

Architecture



Request lifecycle

- 1. **Client** → `POST /api/orchestrate/jobs` with `{ text, cityHint, customerPhone }`. The backend returns a `jobId` immediately and runs the pipeline in the background.
- 2. **Client polls** `GET /api/orchestrate/jobs/:id` every 450 ms, rendering each new `event` in the agent timeline.
- 3. On completion, the result includes `requestUnderstanding`, `recommendation`, `alternatives`, `trace`, and `adapterModes`. The client navigates to the Understanding screen.
- 4. **User confirms** → `POST /api/bookings/:id/confirm` → `sendProviderBookingRequest` fires a Twilio Content Template to the provider's WhatsApp.
- 5. **Provider taps Accept** → Twilio posts to `/api/webhooks/twilio/whatsapp` → reply is parsed → booking state flips to `confirmed` → Firebase realtime channel pushes the update to the app.
- 6. **In-app chat & reminders** continue end-to-end; reschedule / cancel actions also negotiate over WhatsApp.

Agent pipeline

Implemented in `backend/src/services/orchestrator.js`. Each agent records a structured step that's streamed to the client.

#	Agent	File	Tool name	Backing source
1	Intent Understanding	<code>agents/intentAgent.js</code>	<code>parse_request</code>	Gemini 1.5 Flash with strict JSON schema; falls back to a local Roman-Urdu / English regex parser.
2	Location Resolution	<code>agents/locationAgent.js</code>	<code>resolve_location</code>	Google Geocoding API; falls back to a known-sector lookup table for Islamabad / Rawalpindi.
3	Clarification (conditional)	inline	<code>request_missing_details</code>	Triggered when service, location, or time are missing. Returns a <code>needs_input</code> response instead of continuing.
4	Provider Discovery	<code>agents/discoveryAgent.js</code>	<code>find_providers</code>	Google Places Nearby Search filtered by normalized service type.
5	Provider Ranking	<code>agents/rankingAgent.js</code>	<code>rank_providers</code>	Weighted score: availability × proximity × rating × reliability. Uses Google Distance Matrix when configured.
6	Booking	<code>agents/bookingAgent.js</code>	<code>create_booking</code>	Draft booking in <code>pending_user_confirmation</code> ; on user confirm transitions to <code>pending_provider_response</code> and dispatches WhatsApp.
7	Trace Writer	<code>agents/traceAgent.js</code>	<code>write_trace</code>	Persists the full step list with a stable <code>traceId</code> for audit + dev replay.

After dispatch, two reactive components keep the booking moving:

- **Follow-up Agent** (`agents/followUpAgent.js`) — schedules a reminder message and computes the human-readable reminder time once a provider accepts.
- **Provider Reply Parser** (`services/providerReplyParser.js`) — hybrid pipeline triggered on every inbound WhatsApp message:
- Tries the button payload (`ACCEPT`, `REJECT`, `ACCEPT_BK-...`, button title fallback).
- Short replies (≤ 3 words) go to a regex fast-path with Roman-Urdu vocab (`haan`, `nahi`, `theek`, `busy`, ...).
- Longer free-form replies always go to Gemini with a structured-output schema that classifies as `accepted` | `rejected` | `proposed_time` | `unknown` and extracts a verbatim `proposedSlot`.
- Tightened heuristics suppress false positives (`no problem` ≠ rejection; `yes but 4pm instead` = `proposed_time`).

Mock vs. real APIs

Every external dependency is routed through `tools/mode.js → runWithAdapter(name, mockImpl, realImpl)` . The mode is selected per-agent at runtime so a single request can mix real and mock sources during development.

Capability	Real adapter	Mock adapter	Toggle
Intent parsing	Gemini 1.5 Flash (<code>generativelanguage.googleapis.com</code>)	Deterministic Roman-Urdu / English regex parser with synonym tables for ~12 services and Pakistani date/time phrases (<code>kal subah</code> , <code>parson shaam</code>).	<code>GEMINI_API_KEY</code> present + <code>ADAPTER_MODE_INTENT=real</code>
Geocoding	Google Geocoding API	Sector → lat/lng lookup table covering F-series, G-series, I-series sectors of Islamabad and major Rawalpindi areas.	<code>GOOGLE_MAPS_API_KEY</code> + <code>ADAPTER_MODE_LOCATION=real</code>
Provider discovery	Google Places Nearby Search	Synthetic provider catalogue with realistic Pakistani names, ratings, response times, and availability windows.	<code>GOOGLE_MAPS_API_KEY</code> + <code>ADAPTER_MODE_PROVIDER=real</code>
Distance / ranking	Google Distance Matrix	Haversine distance from lat/lng.	<code>ADAPTER_MODE_DISTANCE=real</code>
Booking dispatch	Twilio WhatsApp Business (Content Template)	<code>TWILIO_FAKE_SEND=true</code> produces a <code>SM_FAKE_...</code> SID and stores the rendered body.	Twilio env vars + <code>TWILIO_FAKE_SEND</code>
Reply parsing	Gemini (free-form natural language)	Rule-based regex matcher (<code>haan/naahi/yes/no/accept/reject</code>).	Always uses both; LLM is skipped only if <code>GEMINI_API_KEY</code> is missing.

This is what powers the **"Mock / Real" mode chip** visible on the trace screen — each agent reports the `adapterMode` it actually used so the same demo runs offline at a coffee shop or live in front of an investor.

Integrations

Mobile app

- **Expo Router** for file-based navigation (`app/index.js` , `app/loading.js` , `app/understanding.js` , `app/recommendation.js` , `app/booking.js` , `app/booking-chat.js` , ...).
- **expo-speech-recognition** for on-device voice input with cloud fallback for `ur-PK` ; locale availability is checked via `getSupportedLocales()` and gracefully falls back to `en-US` when Urdu isn't installed. Simulator-aware: refuses to start on iOS Simulator (Apple's `SFSpeechRecognizer` is unstable there).
- **expo-maps** for the location pin on the Understanding screen.
- **React Native Firebase** (`@react-native-firebase/auth` , `app`) for sign-in and the realtime channel that delivers booking-status updates the moment the WhatsApp webhook updates state.
- **expo-notifications** for reminders (push tokens are registered with the backend).

Backend

- **Express** API server (`backend/src/server.js`).
- **Twilio Node SDK** for outbound WhatsApp + inbound webhook signature validation. Content Templates are used for the booking request (so providers get Accept / Reject buttons); plain text bodies are used for chat, reminders, and decision updates — all branded "Karigo" with a consistent header / footer rendering.
- **Google Generative AI REST** for Gemini (structured JSON output via `responseSchema`).
- **Firebase Admin / FCM** for push notifications and the realtime broadcast (`services/realtime.js → emitBookingUpdated`).
- **Local JSON store** (`storage/localStore.js`) — a single `demo-state.json` file with collections (`bookings` , `inboundMessages` , `outboundMessages` , `conversations` , `notifications` , `pushTokens`). Sequence numbers are atomically incremented for IDs.

Webhooks

- `POST /api/webhooks/twilio/whatsapp` — provider replies. Signature-validates with `validateRequest` , pre-matches the booking by extracted `BK-...` ID or by the provider's WhatsApp number against `PROVIDER_TEST_WHATSAPP_TO` , parses intent, applies

state transitions (`pending_provider_response` → `confirmed` / `rejected`), schedules the follow-up reminder, and sends a Karigo confirmation message back to the provider.

- `POST /api/webhooks/twilio/status` — delivery receipts persisted for audit.
 - `POST /api/push-tokens` — registers the user's Expo push token.
-

Data model & storage

`backend/src/storage/localStore.js` exposes `insert` / `list` / `findById` / `updateById` / `nextSequence` over a single JSON file at `backend/data/demo-state.json`. Collections:

- `bookings` — full lifecycle: `pending_user_confirmation` → `pending_provider_response` → `confirmed` | `rejected` | `cancelled`, each transition appended to `statusHistory`.
- `inboundMessages` / `outboundMessages` — every WhatsApp message, with parsed intent and parser used.
- `conversations` — bridge between user and provider, organized by `CONV-<bookingId>`.
- `pushTokens` — per-user FCM/APNs tokens.
- `notifications` — Twilio status callbacks + in-app notification feed.

JSON storage was a deliberate choice to keep the demo reproducible — every state transition is inspectable by opening the file, and the system can be reset to a known state by replacing it.

Repository layout

```

.
├── backend/
│   ├── src/
│   │   ├── server.js                # Express bootstrap
│   │   ├── routes/
│   │   │   └── orchestrate.js       # Orchestration jobs + Twilio webhooks + bookings
│   │   ├── services/
│   │   │   ├── orchestrator.js      # 7-agent pipeline
│   │   │   ├── providerMessaging.js # Twilio outbound + Karigo-branded message builders
│   │   │   ├── providerReplyParser.js # Hybrid rule + Gemini reply classifier
│   │   │   ├── conversationService.js # User ↔ provider bridge
│   │   │   ├── jobStore.js          # Async job + event log
│   │   │   ├── realtime.js          # Firebase broadcast
│   │   │   └── pushNotifications.js
│   │   ├── agents/
│   │   │   ├── intentAgent.js
│   │   │   ├── locationAgent.js
│   │   │   ├── discoveryAgent.js
│   │   │   ├── rankingAgent.js
│   │   │   ├── bookingAgent.js
│   │   │   ├── followUpAgent.js
│   │   │   └── traceAgent.js
│   │   ├── tools/mode.js            # runWithAdapter shim
│   │   └── storage/localStore.js
│   ├── data/demo-state.json
│   ├── test/                        # Jest tests for agents + parser
│   └── .env.local / .env.production
├── mobile-app/
│   ├── app/
│   │   ├── index.js                # Home (voice + categories)
│   │   ├── loading.js              # Live workflow timeline
│   │   ├── understanding.js         # Editable extracted fields
│   │   ├── recommendation.js        # Top provider + alternatives
│   │   ├── booking.js               # Confirmed booking detail
│   │   ├── booking-chat.js          # User ↔ provider thread
│   │   ├── bookings.js / notifications.js / profile.js / login.js
│   │   └── dev/trace.js             # Full agent trace inspector
│   ├── src/
│   │   ├── api.js                  # Backend client
│   │   ├── realtime.js             # Firebase realtime listener
│   │   ├── AuthContext.js
│   │   ├── theme.js
│   │   └── components/
│   └── app.json

```

Running locally

```

# Backend
cd backend
cp .env.example .env.local          # fill in keys (see below)
npm install
npm run dev                          # listens on :4000 by default

# Mobile app (requires Expo CLI + a dev client build)
cd mobile-app
npm install
npx expo prebuild --clean            # native modules (speech recognition, firebase) need this
npx expo run:ios                     # or run:android, or a dev client on a physical device

```

For a fully-offline demo set `TWILIO_FAKE_SEND=true` and leave `ADAPTER_MODE_*` unset (mocks are the default).

Environment variables

Variable	Purpose
GEMINI_API_KEY	Gemini access for intent + reply parsing.
GEMINI_MODEL	Defaults to <code>gemini-1.5-flash</code> .
GOOGLE_MAPS_API_KEY	Geocoding, Places, Distance Matrix.
ADAPTER_MODE_INTENT / _LOCATION / _PROVIDER / _DISTANCE	<code>real</code> or <code>mock</code> per agent.
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN	Twilio credentials.
TWILIO_WHATSAPP_FROM	Karigo's WhatsApp sender number (<code>whatsapp:+14155238886</code> for sandbox).
PROVIDER_TEST_WHATSAPP_TO	Demo provider's number for end-to-end tests.
TWILIO_CONTENT_SID	Approved WhatsApp template SID (gives Accept / Reject buttons). Unset to send the plain Karigo body.
TWILIO_FAKE_SEND	<code>true</code> to skip the real Twilio call and return a fake SID.
TWILIO_VALIDATE_SIGNATURE	<code>true</code> to enforce signature checks on the inbound webhook.
PUBLIC_WEBHOOK_BASE_URL	The publicly reachable URL Twilio should POST back to (your ngrok tunnel in dev).
FIREBASE_*	Service account for Admin SDK + FCM.

What's intentionally simple

- **JSON storage instead of Postgres.** Every booking, message, and notification lives in one file. Trivial to inspect, diff, and reset; obviously not what you'd ship to production.
- **Single demo provider.** `PROVIDER_TEST_WHATSAPP_TO` routes all outbound Twilio messages to one number. The discovery agent still returns many providers — only dispatch is collapsed.
- **Polling instead of streaming.** The client polls `/jobs/:id` every 450 ms rather than holding a WebSocket. Lower complexity, identical UX for sub-second pipelines.
- **No payments.** Out of scope for this build.

These shortcuts make the project demo-stable while leaving every interesting AI/integration boundary fully implemented and observable.